

Keshavai - Architecture & System Guide

Version: 1.0

Date: July 2026

Project: Keshavai AI Chat Platform

Table of Contents

1. [Overview](#1-overview)
 2. [Production Architecture](#2-production-architecture)
 3. [Project Structure](#3-project-structure)
 4. [Chat Message Flow](#4-chat-message-flow)
 5. [AI Model Layer](#5-ai-model-layer)
 6. [Database Schema](#6-database-schema)
 7. [RAG & Vector Search (Neural Schema)](#7-rag--vector-search-neural-schema)
 8. [Memory System](#8-memory-system)
 9. [Authentication & Security](#9-authentication--security)
 10. [Frontend Architecture](#10-frontend-architecture)
 11. [Environment Variables](#11-environment-variables)
 12. [Summary](#12-summary)
-

1. Overview

Keshavai is a production-ready AI chat platform similar to ChatGPT. It supports:

- Multi-provider AI (Gemini, OpenAI, Anthropic, DeepSeek, Ollama)
- Streaming chat responses (SSE)
- RAG (Retrieval-Augmented Generation) with document upload
- Tool calling (calculator, webhooks, etc.)
- JWT authentication with refresh tokens
- User management, usage tracking, subscriptions

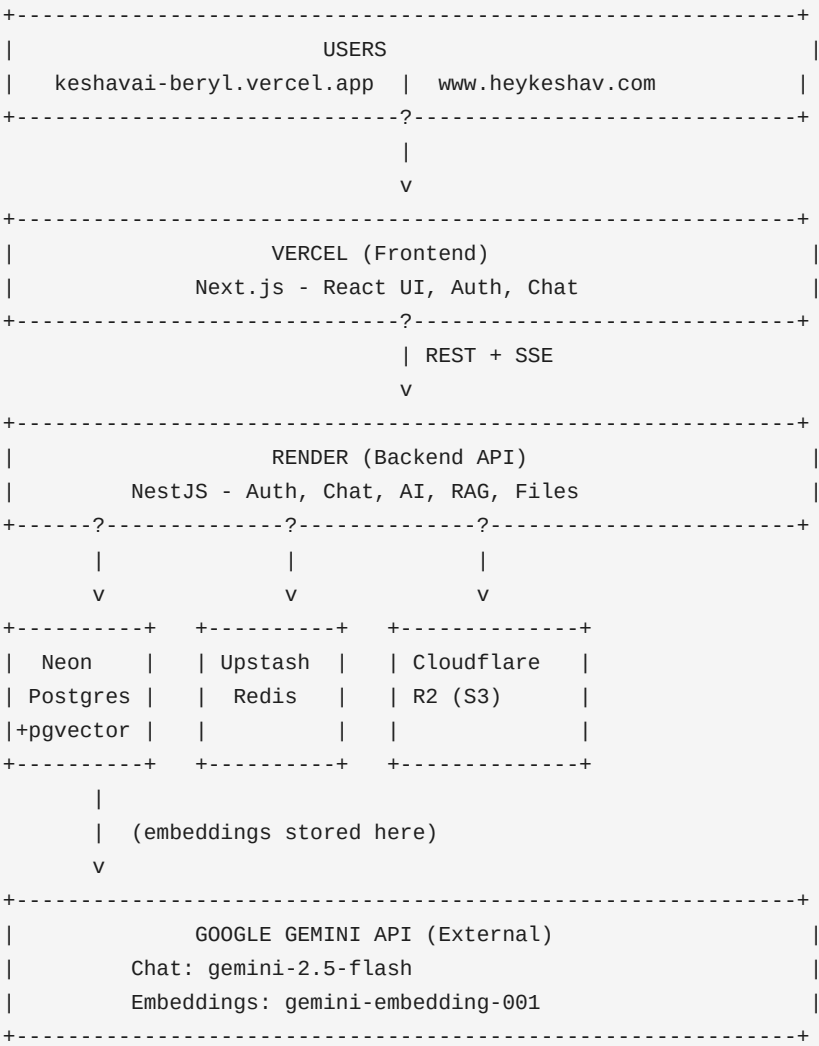
Important: Keshavai does **not** train its own neural network. It calls external AI APIs (e.g. Google Gemini). The only "neural" component you own is **vector storage** for semantic document search (pgvector).

Tech Stack

Layer	Technologies
Frontend	React 19, Next.js 15, TypeScript, TailwindCSS, shadcn/ui, Zustand
Backend	NestJS, Prisma ORM, PostgreSQL, Redis, JWT
AI	OpenAI, Anthropic, Gemini, DeepSeek, Ollama
Storage	S3-compatible (MinIO / Cloudflare R2)
Deploy	Docker, Vercel, Render, Neon, Upstash

2. Production Architecture

Your live deployment:



Service	Role
Vercel	Hosts the Next.js frontend

| **Render** | Runs NestJS API (free tier, sleeps when idle) |

| **Neon** | PostgreSQL database with pgvector extension |

| **Upstash** | Redis for sessions and rate limiting |

| **Cloudflare R2** | Optional file storage for uploads |

| **Google Gemini** | LLM responses + text embeddings |

3. Project Structure

```
keshavai/
+-- frontend/                # Next.js 15 + React 19
|   +-- src/app/              # Pages: /, /login, /register, /settings
|   +-- src/features/chat/    # Chat UI, sidebar, messages, prompt box
|   +-- src/stores/           # Zustand: auth-store, chat-store
|   +-- src/services/         # API client wrappers
|   +-- src/lib/api.ts         # Axios instance + JWT interceptors
|
+-- backend/                  # NestJS API
|   +-- src/
|       +-- modules/
|           | +-- auth/        # Register, login, JWT refresh
|           | +-- chat/        # Conversations + SSE streaming
|           | +-- ai/          # Provider factory + Gemini, OpenAI, etc.
|           | +-- memory/      # Context window + summarization
|           | +-- rag/         # Document upload -> embeddings -> search
|           | +-- tools/       # Calculator, time, webhooks
|           | +-- files/       # S3 storage service
|           | +-- users/       # Profile, usage stats
|       +-- common/           # Guards, filters, decorators
|       +-- config/           # Environment configuration
|       +-- database/         # Prisma service
|
+-- docs/                     # Deployment & architecture docs
+-- docker/                   # Docker & Nginx configs
+-- render.yaml                # Render deployment blueprint
```

4. Chat Message Flow

When a user sends a message, this is the step-by-step flow:

```
User types "hi" in browser
|
v
Frontend: POST /api/v1/chats/:id/messages (SSE stream)
|
v
ChatService.sendMessage()
|
```

```

+-- 1. Save user message -> messages table
|
+-- 2. RAG (optional): embed query -> vector search -> relevant docs
|
+-- 3. MemoryService.buildContext()
|     +-- Load recent messages + chat summary
|
+-- 4. AIService.stream(provider=GEMINI, model, messages)
|     +-- GeminiProvider -> Google Gemini API
|
+-- 5. Stream tokens back via SSE: data: {"content":"..."}
|
+-- 6. Save assistant message + token usage to DB

```

SSE (Server-Sent Events)

The backend streams AI responses in real time:

- Content-Type: `text/event-stream`
- Each chunk: `data: {"content": "partial text"}\n\n`
- Final: `data: {"done": true, "tokenCount": 42}\n\n`

The frontend reads this with `fetch` + `ReadableStream` (not axios).

5. AI Model Layer

Provider Pattern

All AI vendors implement the same interface (`AIProvider`):

```

interface AIProvider {
  readonly name: Provider;      // GEMINI, OPENAI, etc.
  readonly models: string[];    // Available model IDs

  complete(options): Promise<CompletionResult>; // Non-streaming
  stream(options): AsyncGenerator<CompletionChunk>; // Streaming
  embed(text): Promise<number[]>;              // For RAG
}

```

AIProviderFactory

Maps provider enum -> implementation:

Provider	Class	Status in Your Setup
GEMINI	GeminiProvider	**Active** (GEMINI_API_KEY)
OPENAI	OpenAIProvider	Optional

ANTHROPIC	AnthropicProvider	Optional
DEEPSEEK	DeepSeekProvider	Optional
OLLAMA	OllamaProvider	Self-hosted option

Gemini Provider (Your Active Model)

- **Chat:** `generateContentStream()` -> streaming text
- **Embeddings:** `gemini-embedding-001` -> 1536-dim vectors for RAG
- **Default model:** `gemini-2.5-flash`
- **Config:** `GEMINI_API_KEY`, `AI_DEFAULT_PROVIDER=GEMINI`

Configuration Flow

Environment Variables (Render)

```
graph TD
    A[Environment Variables (Render)] --> B[configuration.ts -> app.ai.gemini.apiKey]
    B --> C[GeminiProvider constructor -> GoogleGenerativeAI(apiKey)]
    C --> D[ChatService uses defaultProvider + defaultModel from env]
```

6. Database Schema

PostgreSQL with Prisma ORM. Key entities:

Core Tables

users

- id, email, passwordHash, name, role (USER/ADMIN/MODERATOR)
- preferredModel, preferredProvider, theme

sessions

- refreshToken, userAgent, ipAddress, expiresAt

chats

- id, userId, title, isPinned
- provider (GEMINI, OPENAI, ...), model (gemini-2.5-flash)
- summary (long-term memory)

messages

- chatId, role (USER/ASSISTANT/SYSTEM/TOOL)
- content, model, provider, tokenCount
- toolCalls (JSON), isEdited

****documents****

- userId, fileName, fileType, s3Key
- status (PENDING/PROCESSING/COMPLETED/FAILED)
- chunkCount

****embeddings****

- documentId, content, chunkIndex
- vector (1536 dimensions - pgvector)

****usage****

- userId, provider, model
- promptTokens, completionTokens, totalTokens

****subscriptions****

- userId, plan (FREE/PRO/ENTERPRISE)

Entity Relationships

```
User --1:N--? Chat --1:N--? Message
|
+--1:N--? Document --1:N--? Embedding (vector)
|
+--1:N--? Usage
|
+--1:1--? Subscription
```

7. RAG & Vector Search (Neural Schema)

RAG = Retrieval-Augmented Generation. Lets the AI answer using your uploaded documents.

How It Works

```
UPLOAD PHASE:
PDF/DOCX/TXT uploaded
|
v
Extract text (pdf-parse, mammoth, etc.)
|
v
Split into chunks (~1000 chars, 200 overlap)
```

```

      |
      v
Gemini embed each chunk -> 1536-number vector
      |
      v
Store in embeddings table (content + vector)

QUERY PHASE:
  User asks "What does my document say about X?"
      |
      v
  Embed the question -> vector
      |
      v
  PostgreSQL pgvector: cosine similarity search
      |
      v
  Top 5 most similar chunks retrieved
      |
      v
  Chunks injected into system prompt
      |
      v
  Gemini generates answer with document context

```

Embedding Table (Neural Storage)

Column	Type	Purpose
id	UUID	Primary key
document_id	UUID	Links to source document
content	TEXT	Original text chunk
chunk_index	INT	Order in document
vector	vector(1536)	Semantic embedding from Gemini

****Search query (simplified):****

```

SELECT content, 1 - (vector <=> $query_vector) AS score
FROM embeddings e
JOIN documents d ON d.id = e.document_id
WHERE d.user_id = $userId AND d.status = 'COMPLETED'
ORDER BY vector <=> $query_vector
LIMIT 5

```

Similar meaning -> vectors close together -> relevant context for the LLM.

8. Memory System

`MemoryService` manages conversation context within token limits.

Short-Term Memory

- Loads recent messages from the chat
- Fits within ~8000 token budget
- Most recent messages prioritized

Long-Term Memory (Summarization)

- When chat exceeds 20 messages
- Older history summarized via AI
- Summary stored in `chats.summary`
- Included as system message in future requests

Context Building

```
System prompt: "You are a helpful AI assistant."
      +
Optional RAG context from documents
      +
Chat summary (if exists): "Previous conversation summary: ..."
      +
Recent messages (within token limit)
      +
Current user message
      |
      v
Sent to Gemini API
```

9. Authentication & Security

Login Flow

```
POST /auth/login { email, password }
      |
      v
Verify bcrypt password hash
      |
      v
Return { accessToken (15min), refreshToken (7 days) }
      |
      v
Frontend stores in localStorage
      |
      v
All API requests: Authorization: Bearer <accessToken>
      |
```


v

On 401: auto-refresh via POST /auth/refresh

Guards

- ****JwtAuthGuard**** - Protects all routes except @Public()
- ****RolesGuard**** - RBAC (USER, ADMIN, MODERATOR)
- ****ThrottlerGuard**** - Rate limiting via Redis

CORS (Multi-Domain)

- ``CORS_ORIGINS`` - Explicit allowed URLs
- ``CORS_ALLOWED_HOSTS`` - e.g. heykeshav.com (allows www + apex)
- ``*.vercel.app`` - Auto-allowed for preview deployments

10. Frontend Architecture

Pages

| Route | Purpose |

|-----|-----|

| ``/`` | Main chat interface |

| ``/login`` | Sign in |

| ``/register`` | Create account |

| ``/settings`` | Profile, AI model selection, usage stats |

State Management (Zustand)

****auth-store****

- user, accessToken, refreshToken
- setAuth(), logout()

****chat-store****

- chats[], activeChatId
- isStreaming, streamingContent
- selectedProvider, selectedModel

API Client

- Base URL: ``NEXT_PUBLIC_API_URL`` (Render backend)
- Axios with JWT interceptors

- Chat streaming uses raw fetch for SSE

11. Environment Variables

Render (Backend)

Variable	Example	Purpose
-----	-----	-----
DATABASE_URL	postgresql://...@neon.tech/...	PostgreSQL
REDIS_URL	rediss://...@upstash.io:6379	Redis
JWT_SECRET	random-32-char-string	Access token signing
JWT_REFRESH_SECRET	random-string	Refresh token signing
GEMINI_API_KEY	AIza... or AQ....	Google Gemini API
AI_DEFAULT_PROVIDER	GEMINI	Default AI provider
AI_DEFAULT_MODEL	gemini-2.5-flash	Default model
FRONTEND_URL	https://www.heykeshav.com	Primary frontend URL
CORS_ORIGINS	https://keshavai-beryl.vercel.app,...	Allowed origins
CORS_ALLOWED_HOSTS	heykeshav.com	Custom domain CORS

Vercel (Frontend)

Variable	Example	Purpose
-----	-----	-----
NEXT_PUBLIC_API_URL	https://keshavai-chq6.onrender.com/api/v1	Backend API base

12. Summary

Question	Answer
-----	-----
What is Keshavai?	ChatGPT-style platform with multi-provider AI, RAG, tools, auth
Where does intelligence live?	External APIs (Gemini) - not on your server
What is the "neural schema"?	`embeddings` table with pgvector for semantic search
How does chat work?	Message -> memory + RAG -> provider stream -> SSE -> DB
Your production stack	Vercel + Render + Neon + Upstash + Gemini
Custom domains	heykeshav.com + keshavai-beryl.vercel.app

Appendix: API Endpoints (Key Routes)

| Method | Path | Description |

|-----|-----|-----|

| POST | /auth/register | Create account |

| POST | /auth/login | Sign in |

| POST | /auth/refresh | Refresh JWT |

| GET | /chats | List user chats |

| POST | /chats | Create chat |

| GET | /chats/:id | Get chat + messages |

| POST | /chats/:id/messages | Send message (SSE stream) |

| GET | /ai/providers | List AI providers & models |

| GET | /users/me | User profile |

| GET | /users/me/usage | Token usage stats |

****Swagger docs:**** ``https://your-backend.onrender.com/api/docs``

Document generated for Keshavai project - technolitics/keshavai